

# Supplementary Material for The Generator–Verifier Gap Predicts When Multi-Agent LLM Architectures Beat a Compute-Matched Single Agent

Madhavam Pratap Shahi  
Independent Researcher  
madhavam.shahi.12@gmail.com

Kavish Soningra  
Independent Researcher

Nagendra Chaudhary  
Independent Researcher

Section, equation, table and figure numbers cited below refer to the main paper. The code, the frozen task suite and every cached generation are at <https://github.com/madhavamshahi/generator-verifier-gap>.

## APPENDIX A REPRODUCIBILITY

Every result in this paper was produced by the code released alongside it, on a single Apple M4 Pro (24 GB unified memory) with no API access and no paid compute. The pipeline is:

- 1) `gvgap/build_data.py` — builds the frozen 540-task suite. Synthetic families are generated from seed 20260919; benchmark families are subsampled with the same seed. The calibration/held-out split is assigned here, before any model runs.
- 2) `gvgap/run.py` — runs all architectures for all models. Every generation is cached by a hash of (model, prompt, max tokens, temperature, seed), so the experiment is resumable and re-running it is free.
- 3) `gvgap/analysis.py`, `predict.py`, `report.py`, `figures.py` — aggregation, hypothesis tests, tables and figures.

Total generations: 21,847. Models are Qwen2.5-Instruct at 0.5B and 1.5B [1] in `bfloat16` via `MLX`; `gvgap/run.py` runs a 3B scale point unchanged by uncommenting one line.

*a) Estimator validation.*: The estimators the paper’s claims rest on are tested against ground truth rather than against themselves (`gvgap/test_metrics.py`): `pass@k` is checked against exhaustive enumeration of all  $k$ -subsets for every  $n \leq 8$ ; the bootstrap interval is checked for empirical coverage (0.963 against a nominal 0.95); the paired bootstrap is checked for its false-positive rate under the null (0.057 at  $\alpha = 0.05$ ); and the latent-score model is checked against its own boundary conditions, including that it reduces exactly to uniform selection ( $c/k$ ) when  $G = 0$  and is monotone in  $G$ .

*b) Bibliography hygiene.*: Every reference in this paper was resolved against the arXiv API before inclusion, and

entries are generated from the returned metadata rather than transcribed. This caught four incorrect titles in our own first draft of the reference list — an error rate of roughly one in six — which is a small illustration of the case study’s point in Section ??: a cheap external check finds errors that careful reading does not.

## APPENDIX B TASK FAMILY DETAILS

*a) CSP.*: Moduli are drawn from  $\{3, \dots, 9\}$  with  $k = 2$  congruences and lower bounds in  $\{10, 20, 30, 50, 100\}$ . The answer is verified structurally: we re-check the congruences and confirm minimality by an upward scan, rather than trusting the stored answer.

*b) Logic.*: 4–6 entities are placed in a total order; the  $n - 1$  adjacent comparisons are emitted in random surface form (“ $X$  is taller than  $Y$ ” or “ $Y$  is shorter than  $X$ ”) and shuffled. The question asks for the maximum or minimum.

*c) WordCon.*: Target lengths are 6–9 words with two required content words drawn from a fixed topic list. Grading tokenises on `[A-Za-z’ ]+` and requires an exact length match plus both required words as case-insensitive substrings.

*d) MBPP.*: We use the full test split, shuffled with the suite seed. Candidates are executed in a subprocess with an 8-second timeout. The selector may use only the single test shown in the prompt; grading uses the remaining tests.

## APPENDIX C PROMPTS

All architectures share one system prompt per family, so that differences are attributable to orchestration rather than prompt engineering. The judge is shown candidates in an order permuted per task with a fixed seed, and the permutation is recorded so that position bias can be measured separately (reported in Table ??). The pairwise probe presents exactly one correct and one incorrect candidate in randomised order. Full prompt text is in `gvgap/prompts.py`.

## APPENDIX D

### FIGURE DESIGN

Categorical colours are assigned in a fixed order so that an architecture carries the same hue in every figure, and the palette was checked programmatically for colour-vision-deficiency separation (worst adjacent pair  $\Delta E = 9.1$  in OKLab $\times 100$ , above the threshold of 8) rather than by eye. Three hues fall below a 3:1 contrast ratio against the page, so every figure also carries axis labels and a corresponding table, and identity is never carried by colour alone.

### REFERENCES

- [1] Qwen *et al.*, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.